

An animated AI news anchor that charts stocks, explains the market out loud, takes voice commands and exports reports — with every answer inferred **100% locally on an AMD Radeon GPU via ROCm**. Your questions, watchlist and portfolio **never leave the machine**.

WHY PRIVATE-BY-ARCHITECTURE

AI market tools ship your financial curiosity — every question, ticker and position — to a cloud API. Vantage inverts that: the whole agent (reasoning, tool use, memory, report writing) runs against a local open-weights model. One URL flag (?local=1) disables every cloud path.

AGENT CAPABILITIES

Tool invocation

Charts, watchlist & portfolio ops, navigation, market calendar with on-air alerts — from plain language or voice.

Local multi-turn memory

Last 6 exchanges kept on-device; "what about its risks?" resolves against earlier turns. One-click *forget conversation*.

Privacy & permission control

Accounts + plan gates; local-only mode; memory clearing; zero external calls in demo data mode.

Report writer

"Write a report and export ppt" → local model writes it, browser builds PowerPoint / Word / Excel. No server.

EXPERIENCE

Animated anchor reads answers on air (TTS) · voice input · 6 languages · 18 studio sets · live or demo tape.

ARCHITECTURE — ONE QUESTION, END TO END

One input box (typed or voice)

"chart AMD and explain the move" · "add TSLA" · "write a report and export ppt"

askDesk() — intent router (our open code, no agent platform)

Export / navigation / alert intents run tools directly; open questions continue to the model

Prompt assembly

Question + compact market-snapshot JSON + local conversation memory (6 exchanges, raw Q/A, localStorage)

AMD Radeon GPU · ROCm — Ollama (llama.cpp) / vLLM

Llama 3.1 8B Q4_K_M, fully GPU-resident · streamed tokens · OpenAI-compatible local API · no cloud keys

Anchor answers on air

Streaming render + speech; exchange written back to local memory; tools fire (chart, deck, alert)

```
$ ollama ps
NAME          SIZE  PROCESSOR
llama3.1:latest 5.3 GB 100% GPU ✓ ← verified on the Radeon instance,
rocm-smi on video
```

INFERENCE-SPEED OPTIMIZATION

- Q4_K_M quantization** — 8B in ~4.9 GB: whole model GPU-resident, no CPU offload split
- Bounded prompts** — memory capped at 6 raw Q/A turns; snapshot is compact JSON → prefill stays constant
- Streaming** — perceived latency ≈ time-to-first-token; the anchor starts speaking mid-decode
- Right-size switch** — one-line swap to a 1B model ≈ halves wall-clock for latency-critical use

Setup (on Radeon / ROCm)	Answer time
Llama 3.1 8B Q4_K_M	live ▶ video
Llama 3.2 1B	live ▶ video
Memory-recall turn (multi-turn)	live ▶ video

Latency is demonstrated live on the Radeon instance in the demo video — rocm-smi + ollama ps at 100% GPU on camera, measured not asserted.

REPRODUCE IT

```
bash scripts/radeon-setup.sh
→ http://127.0.0.1:5173/?local=1
ollama ps # must read 100% GPU
```

Open stack: React SPA · Ollama (llama.cpp) / vLLM on ROCm · open-weights Llama · zero cloud dependencies in local mode. Works fully offline.